```
    HttpClient client = factory.CreateClient("rates");          ❻

    var response = await client.GetAsync("latest");             ❼
                                                                ❼
    response.EnsureSuccessStatusCode();
    return await response.Content.ReadAsStringAsync();
});
app.Run();
```

❶ Provides a name for the client and a configuration function

❷ The configuration function runs every time the named HttpClient is requested.

❸ You can add more configuration functions for the named client, which run in sequence.

❹ Additional overloads exist that allow access to the DI container when creating a named client.

❺ Injects the IHttpClientFactory using DI

❻ Requests the configured named client called “rates”

❼ Uses the HttpClient the same way as before

NOTE You can still create unconfigured clients using `CreateClient()` without a name. Be aware that if you pass an unconfigured name, such as `CreateClient ("MyRates")`, the client returned will be unconfigured. Take care—client names are case-sensitive, so `"rates"` is a different client from `"Rates"`.

Named clients help centralize your `HttpClient` configuration in one place, removing the responsibility for configuring the client from your consuming code. But you’re still working with raw HTTP calls at this point, such as providing the relative URL to call (`"/latest"`) and parsing the response. `IHttpClientFactory` includes a feature that makes it easier to clean up this code.

33.2.3 Using typed clients to encapsulate HTTP calls

A common pattern when you need to interact with an API is to encapsulate the mechanics of that interaction in a separate service. You could easily do